

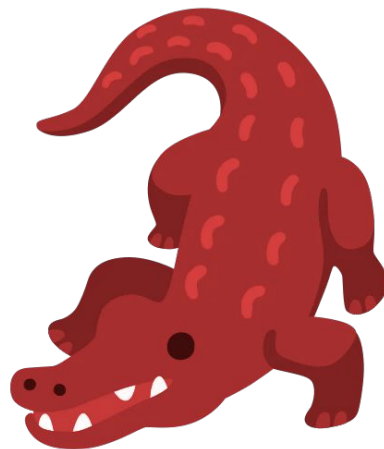
TurboGator

Adam Suma

Frederico Aberle

Jannis Alsbach

Mihai-George Licu



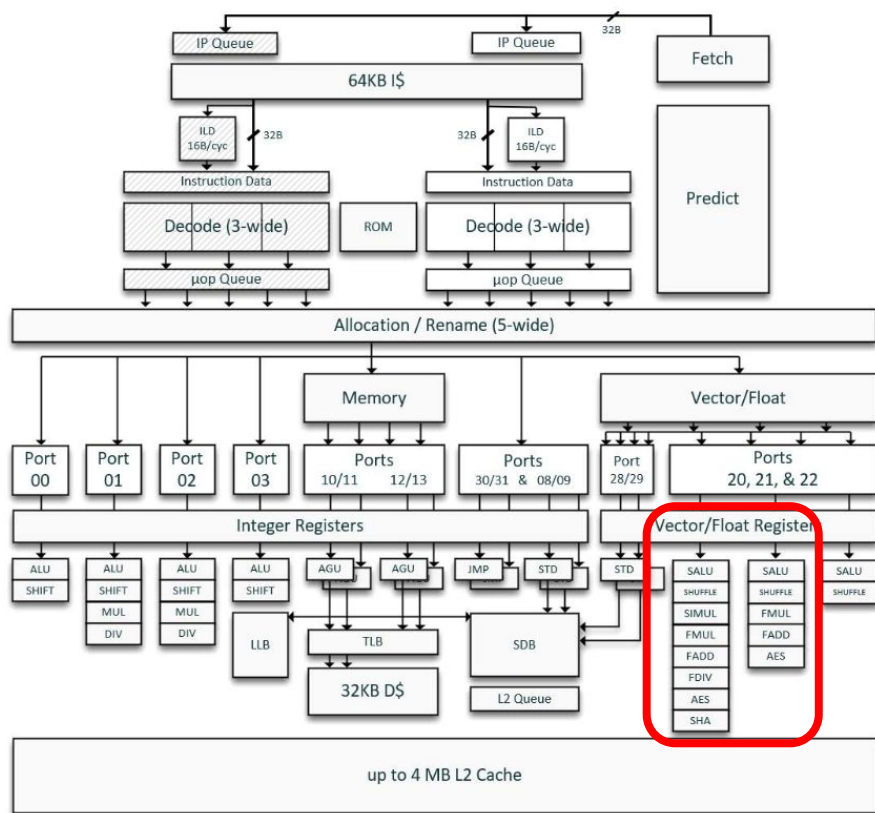
ETH

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Todo Slides:

- Bottlenecks fully identified
- Bottlenecks optimized
 - Sparsity
 - Simplicity
 - Memory Layout Optimizations
 - Flattening & Contiguous vectors
 - Blocking ?
 - Others ?
- Memory layout optimization
- Runtime & roofline plots
- (BOLD) TODO Vectorization

Recap Server setup



Intel N100 Alder Lake E core (Gracemont)

Intel SSE4.1, Intel SSE4.2, Intel AVX2, FMA3

TurboBoost off 800MHz, 12 GB RAM

256b instructions get "cracked" in two 128b instruction

$\pi_s = 3.33$ flops/cycle

$\pi_v = 16$ flops/cycle

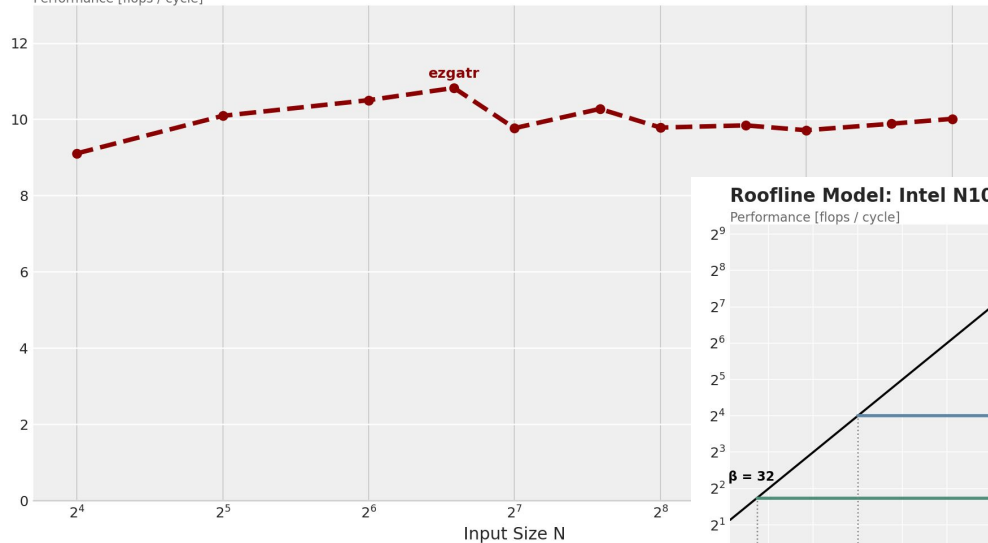
$\beta = 32$ bytes/cycle

L1d = 32 KiB; Latency 3 Cycles

Last time on TurboGator...

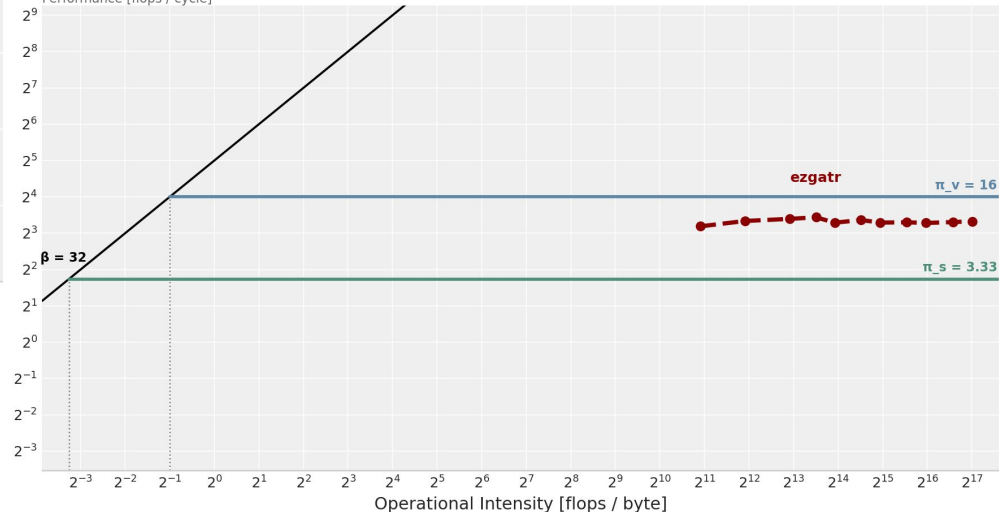
Performance: Intel N100 (1 Core, 800Mhz)

Performance [flops / cycle]



Roofline Model: Intel N100 (1 Core, 800Mhz)

Performance [flops / cycle]



New sweep

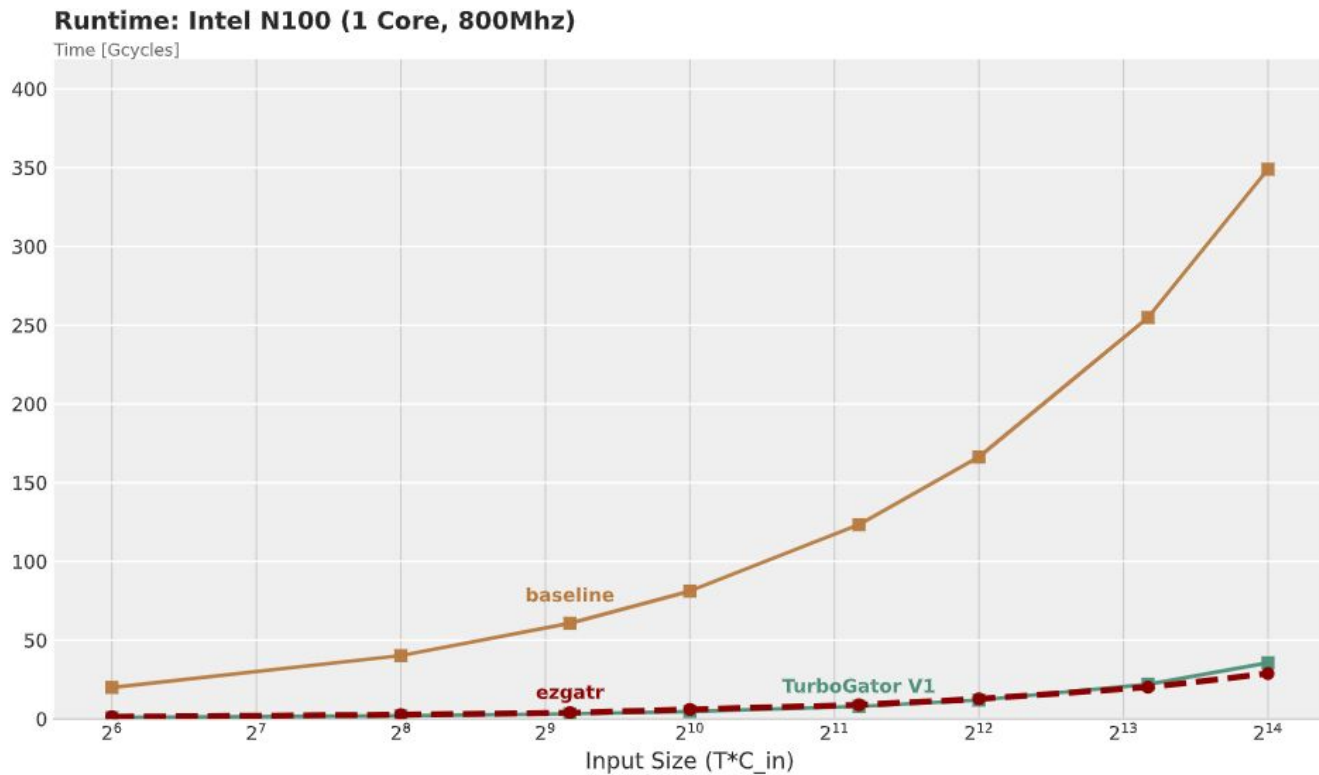
$$X = (8, T, C_{in}, 16)$$

$$T = 32 * N$$

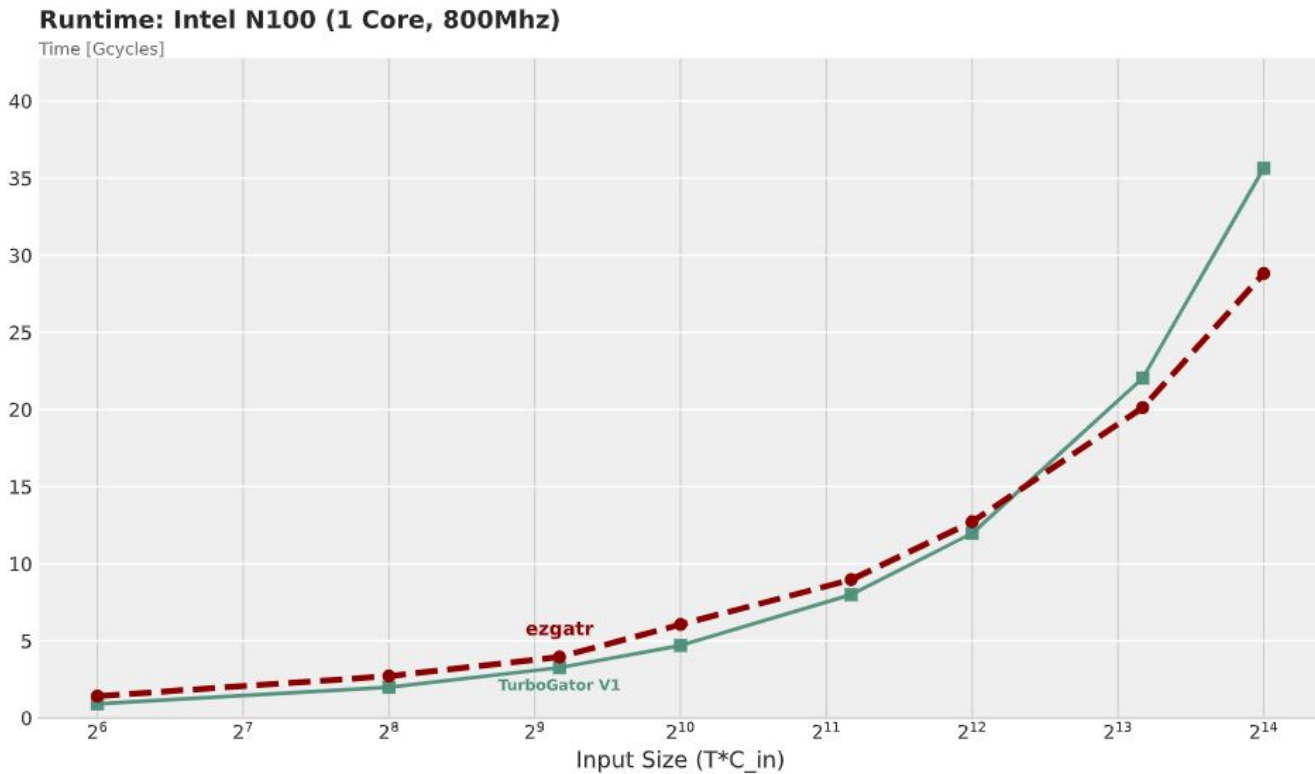
$$C_{in} = 2 * N$$

N=1 -> (32, 2)	= 64
N=2 -> (64, 4)	= 256
N=3 -> (96, 6)	= 576
N=4 -> (128, 8)	= 1024
N=6 -> (192, 12)	= 2304
N=8 -> (256, 16)	= 4096
N=12 -> (384, 24)	= 9216
N=16 -> (512, 32)	= 16384

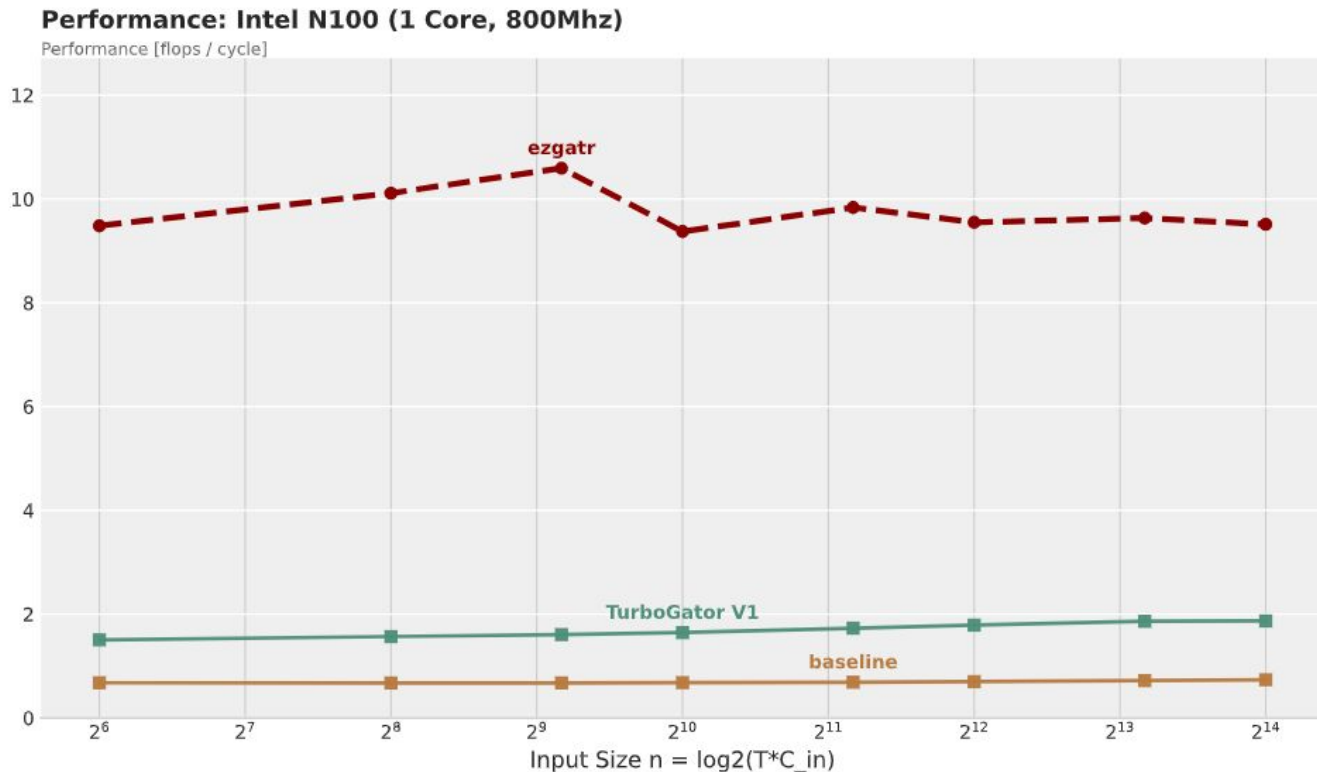
Runtime Today



Runtime Today (ignore baseline)



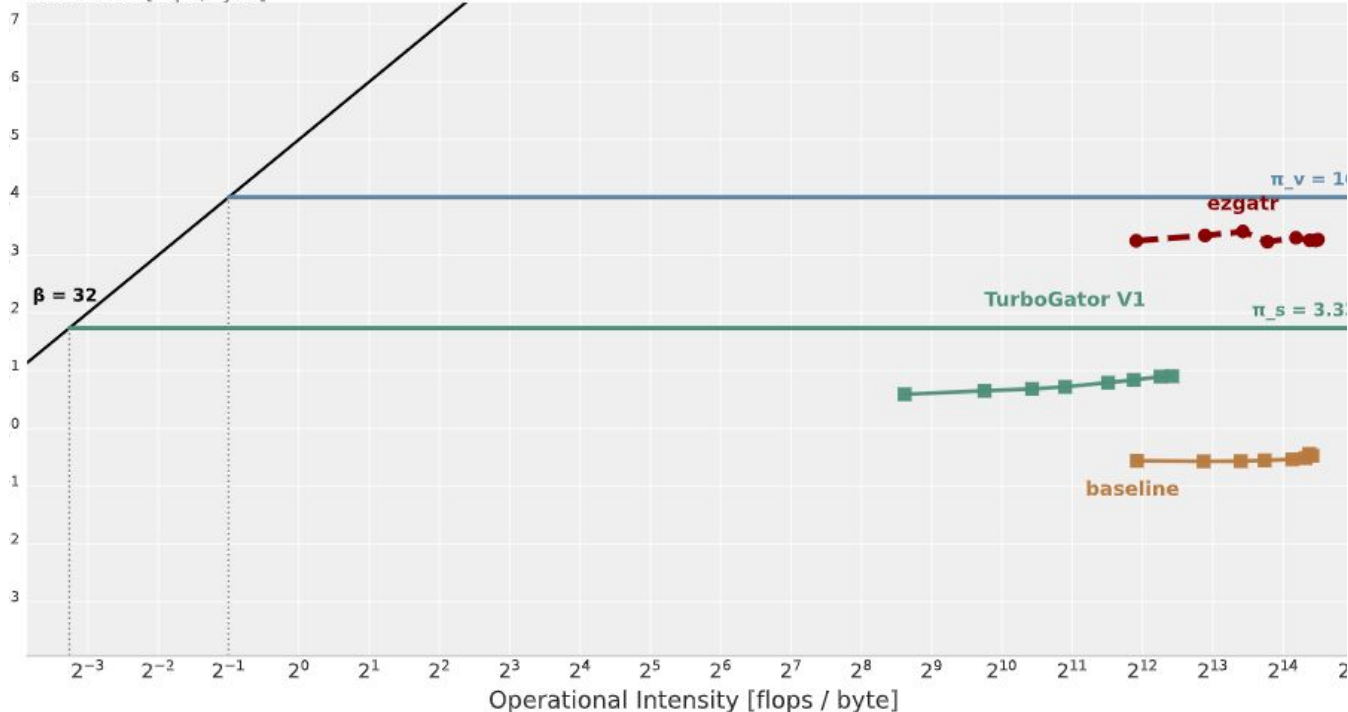
Performance Today



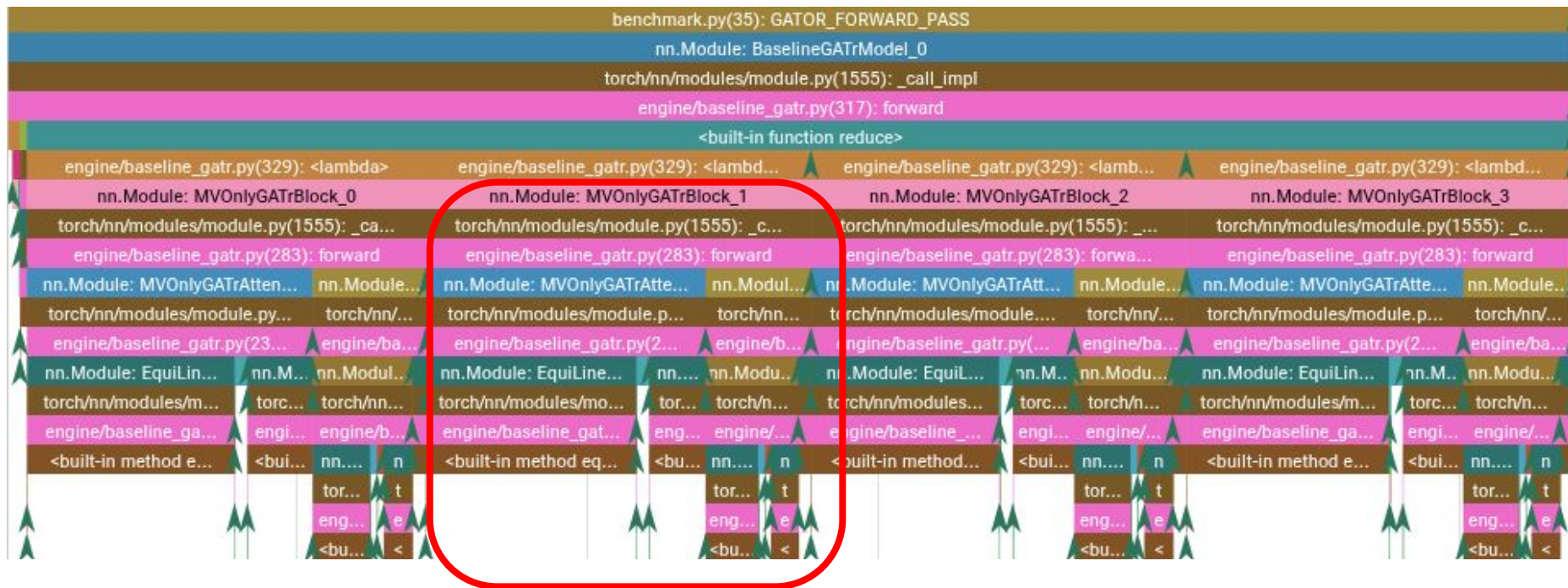
Roofline Today

Roofline Model: Intel N100 (1 Core, 800Mhz)

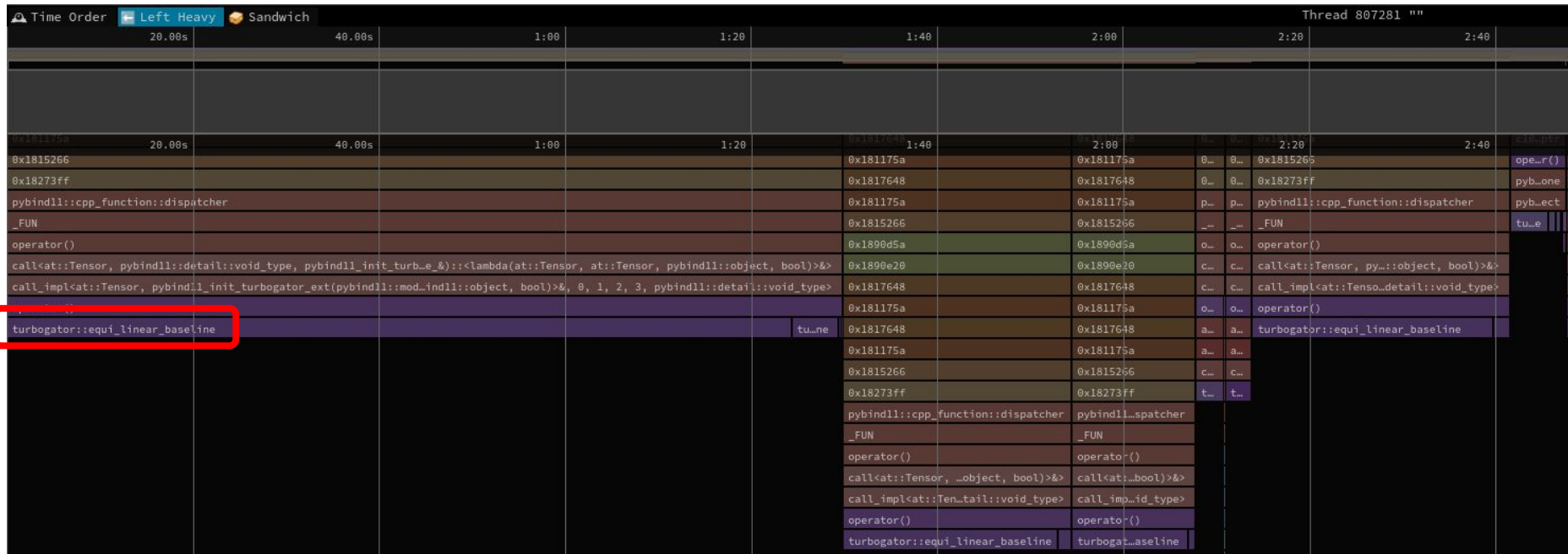
Performance [flops / cycle]



Profiling (pytorch Profiler)



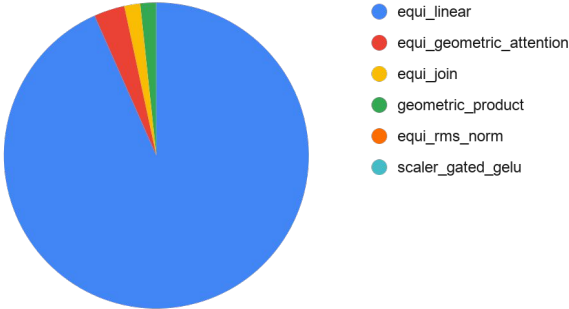
Profiling (py-spy)



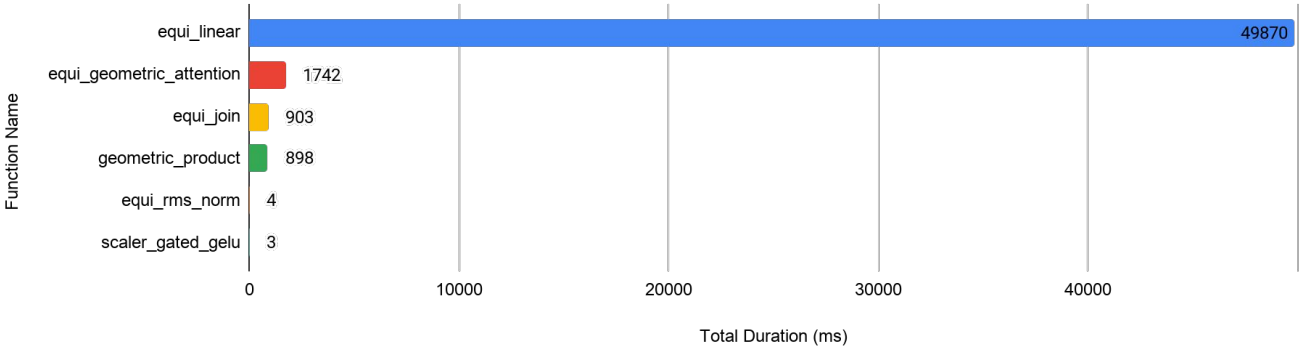
Bottlenecks identified

Function Name	Call Count	Total Duration (ms)	% of Total Trace
equi_linear	22	49870	92
equi_geometric_attention	4	1742	3
equi_join	4	903	1
geometric_product	4	898	1
equi_rms_norm	8	4	0.007
scaler_gated_gelu	4	3	0.006

Distribution of Trace Time by Function



Total Duration by Baseline Function



Bottleneck Optimizations

- Simplicity
- Scalar Replacement
- Sparsity
- Memory Layout Optimizations
 - Flattening & Contiguous vectors
 - Blocking
- Vectorization with AVX (coming soon)

Simplicity

```
for (int i = 0; i < 4; ++i)
    for (int j = 0; j < 4; ++j)
        for (int k = 0; k < N_DAA; ++k)
            out[k] += basis[i][j][k] * r[i] * r[j];
```



```
out[0] = -(r[3]*r[3]);
out[1] = -(r[0]*r[0] + r[1]*r[1] + r[2]*r[2]);
out[2] = 2.f * r[0]*r[3];
out[3] = 2.f * r[1]*r[3];
out[4] = 2.f * r[2]*r[3];
```

Scalar replacements

```
// step 2: out[b,o,d] = sum_{i,s} kernel[o,i,d,s] * x[b,i,s]
for (size_t b = 0; b < batch; ++b)
{
    for (size_t oc = 0; oc < out_channels; ++oc)
    {
        for (size_t d = 0; d < 16; ++d)
        {
            float sum = 0.0f;
            for (size_t ic = 0; ic < in_channels; ++ic)
            {
                for (size_t s = 0; s < 16; ++s)
                {
                    sum += kernel[(oc * in_channels + ic) * 16 + d] * 16 + s]
                        * x[b * in_channels * 16 + ic * 16 + s];
                }
            }
            out[b * out_channels * 16 + oc * 16 + d] = sum;
        }
    }
}
```



```
// step 2: out[b,o,d] = sum_{ic,s} kernel[o,ic,s,d] * x[b,ic,s]
// OPT1:
// - register accumulator: acc[16] holds all d values for one (b, oc)
// - x[b,ic,s] is loaded once per (ic,s) and reused across all 16 d
for (size_t b = 0; b < batch; ++b)
{
    for (size_t oc = 0; oc < out_channels; ++oc)
    {
        float acc[16] = {0.0f};
        for (size_t ic = 0; ic < in_channels; ++ic)
        {
            for (size_t s = 0; s < 16; ++s)
            {
                float xv = x[b * in_channels * 16 + ic * 16 + s];
                for (size_t d = 0; d < 16; ++d)
                {
                    acc[d] += kernel[(oc * in_channels + ic) * 16 + s] * 16 + d] * xv;
                }
            }
        }
        for (size_t d = 0; d < 16; ++d)
        {
            out[b * out_channels * 16 + oc * 16 + d] = acc[d];
        }
    }
}
```


Sparsity

```
float data[16][16][16] = {};  
LocalGPBasis()  
{  
    data[0][0][0] = 1.0f;  
    data[0][2][2] = 1.0f;  
    data[0][3][3] = 1.0f;  
    data[0][4][4] = 1.0f;  
    data[0][8][8] = -1.0f;  
    data[0][9][9] = -1.0f;  
    data[0][10][10] = -1.0f;  
    data[0][14][14] = -1.0f;  
    data[1][0][1] = 1.0f;  
    ...  
}
```

95% reduction 🤨

```
static constexpr GPEntry data[] = {  
    {0, 0, 0, 1}, {0, 2, 2, 1}, {0, 3, 3, 1}, {0, 4, 4, 1},  
    {0, 8, 8, -1}, {0, 9, 9, -1}, {0, 10, 10, -1}, {0, 14, 14, -1},  
    {1, 0, 1, 1}, {1, 1, 0, 1}, {1, 2, 5, -1}, {1, 3, 6, -1},  
    {1, 4, 7, -1}, {1, 5, 2, 1}, {1, 6, 3, 1}, {1, 7, 4, 1},  
    {1, 8, 11, -1}, {1, 9, 12, -1}, {1, 10, 13, -1}, {1, 11, 8, -1},  
    {1, 12, 0, -1}, {1, 13, 10, -1}, {1, 14, 15, 1}, {1, 15, 14, -1},  
    {2, 0, 2, 1}, {2, 2, 0, 1}, {2, 3, 8, -1}, {2, 4, 9, -1},  
    {2, 8, 3, 1}, {2, 9, 4, 1}, {2, 10, 14, -1}, {2, 14, 10, -1},  
    {3, 0, 3, 1}, {3, 2, 8, 1}, {3, 3, 0, 1}, {3, 4, 10, -1},  
    {3, 8, 2, -1}, {3, 9, 14, 1}, {3, 10, 4, 1}, {3, 14, 9, 1},  
    {4, 0, 4, 1}, {4, 2, 9, 1}, {4, 3, 10, 1}, {4, 4, 0, 1},  
    {4, 8, 14, -1}, {4, 9, 2, -1}, {4, 10, 3, -1}, {4, 14, 8, -1},  
    {5, 0, 5, 1}, {5, 1, 2, 1}, {5, 2, 1, -1}, {5, 3, 11, 1},  
    {5, 4, 12, 1}, {5, 5, 0, 1}, {5, 6, 8, -1}, {5, 7, 9, -1},  
    {5, 8, 6, 1}, {5, 9, 7, 1}, {5, 10, 15, -1}, {5, 11, 3, 1},  
    {5, 12, 4, 1}, {5, 13, 14, -1}, {5, 14, 13, 1}, {5, 15, 10, -1},  
    {6, 0, 6, 1}, {6, 1, 3, 1}, {6, 2, 11, -1}, {6, 3, 1, -1},  
    {6, 4, 13, 1}, {6, 5, 8, 1}, {6, 6, 0, 1}, {6, 7, 10, -1},  
    {6, 8, 5, -1}, {6, 9, 15, 1}, {6, 10, 7, 1}, {6, 11, 2, -1},  
    {6, 12, 14, 1}, {6, 13, 4, 1}, {6, 14, 12, -1}, {6, 15, 9, 1},  
    {7, 0, 7, 1}, {7, 1, 4, 1}, {7, 2, 12, -1}, {7, 3, 13, -1},  
    {7, 4, 1, -1}, {7, 5, 9, 1}, {7, 6, 10, 1}, {7, 7, 0, 1},  
    {7, 8, 15, -1}, {7, 9, 5, -1}, {7, 10, 6, -1}, {7, 11, 14, -1},  
    {7, 12, 2, -1}, {7, 13, 3, -1}, {7, 14, 11, 1}, {7, 15, 8, -1},  
    {8, 0, 8, 1}, {8, 2, 3, 1}, {8, 3, 2, -1}, {8, 4, 14, 1},  
    {8, 8, 0, 1}, {8, 9, 10, -1}, {8, 10, 9, 1}, {8, 14, 4, 1},  
    {9, 0, 9, 1}, {9, 2, 4, 1}, {9, 3, 14, -1}, {9, 4, 2, -1},  
    {9, 8, 10, 1}, {9, 9, 0, 1}, {9, 10, 8, -1}, {9, 14, 3, -1},  
    {10, 0, 10, 1}, {10, 2, 14, 1}, {10, 3, 4, 1}, {10, 4, 3, -1},  
    {10, 8, 9, -1}, {10, 9, 8, 1}, {10, 10, 0, 1}, {10, 14, 2, 1},  
    {11, 0, 11, 1}, {11, 1, 8, 1}, {11, 2, 6, -1}, {11, 3, 5, 1},  
    {11, 4, 15, -1}, {11, 5, 3, 1}, {11, 6, 2, -1}, {11, 7, 14, 1},  
    {11, 8, 1, 1}, {11, 9, 13, -1}, {11, 10, 12, 1}, {11, 11, 0, 1},  
    {11, 12, 10, -1}, {11, 13, 9, 1}, {11, 14, 7, -1}, {11, 15, 4, 1},  
    {12, 0, 12, 1}, {12, 1, 9, 1}, {12, 2, 7, -1}, {12, 3, 15, 1},  
    {12, 4, 5, 1}, {12, 5, 4, 1}, {12, 6, 14, -1}, {12, 7, 2, -1},  
    {12, 8, 13, 1}, {12, 9, 1, 1}, {12, 10, 11, -1}, {12, 11, 10, 1},  
    {12, 12, 0, 1}, {12, 13, 8, -1}, {12, 14, 6, 1}, {12, 15, 3, -1},  
    {13, 0, 13, 1}, {13, 1, 10, 1}, {13, 2, 15, -1}, {13, 3, 7, -1},  
    {13, 4, 6, 1}, {13, 5, 14, 1}, {13, 6, 4, 1}, {13, 7, 3, -1},  
    {13, 8, 12, -1}, {13, 9, 11, 1}, {13, 10, 1, 1}, {13, 11, 9, -1},  
    {13, 12, 8, 1}, {13, 13, 0, 1}, {13, 14, 5, -1}, {13, 15, 2, 1},  
    {14, 0, 14, 1}, {14, 2, 10, 1}, {14, 3, 9, -1}, {14, 4, 8, 1},  
    {14, 8, 4, 1}, {14, 9, 3, -1}, {14, 10, 2, 1}, {14, 14, 0, 1},  
    {15, 0, 15, 1}, {15, 1, 14, 1}, {15, 2, 13, -1}, {15, 3, 12, 1},  
    {15, 4, 11, -1}, {15, 5, 10, 1}, {15, 6, 9, -1}, {15, 7, 8, 1},  
    {15, 8, 7, 1}, {15, 9, 6, -1}, {15, 10, 5, 1}, {15, 11, 4, 1},  
    {15, 12, 3, -1}, {15, 13, 2, 1}, {15, 14, 1, -1}, {15, 15, 0, 1},  
};
```


Sparsity II

```
float unnorm_basis[9][16][16] = {};  
float norm_basis[9][16][16] = {};  
  
LocalEquiLinearBasis()  
{  
    const std::vector<std::vector<BasisElement>> basis_elements = {  
        {0},  
        {1, 2, 3, 4},  
        {5, 6, 7, 8, 9, 10},  
        {11, 12, 13, 14},  
        {15},  
        {std::pair<int, int>{1, 0}},  
        {std::pair<int, int>{5, 2}, std::pair<int, int>{6, 3}, std::pair<int, int>{7, 4}},  
        {std::pair<int, int>{11, 8}, std::pair<int, int>{12, 9}, std::pair<int, int>{13, 10}},  
        {std::pair<int, int>{15, 14}},  
    };  
};
```

Sparsity II

```
for (size_t b = 0; b < batch; ++b)
{
    for (size_t oc = 0; oc < out_channels; ++oc)
    {
        float acc[16] = {0.0f};

        for (size_t ic = 0; ic < in_channels; ++ic)
        {
            for (size_t s = 0; s < 16; ++s)
            {
                float xv = x[b * in_channels * 16 + ic * 16 + s];
                for (size_t d = 0; d < 16; ++d)
                {
                    acc[d] += kernel[(oc * in_channels + ic) * 16 + s] * xv;
                }
            }

            for (size_t d = 0; d < 16; ++d)
            {
                out[b * out_channels * 16 + oc * 16 + d] = acc[d];
            }
        }
    }
}
```

256 $\rightarrow$ 26 FMAs per
(b, oc, ic)



```
for (size_t ic = 0; ic < in_channels; ++ic)
{
    const float *w_oi = w_use.data() + (oc * in_channels + ic) * 9;
    const float *x_bi = x + (b * in_channels + ic) * 16;

    // load weights for (oc, ic)
    const float w0 = w_oi[0];
    const float w1 = w_oi[1];
    const float w2 = w_oi[2];
    const float w3 = w_oi[3];
    const float w4 = w_oi[4];
    const float w5 = w_oi[5];
    const float w6 = w_oi[6];
    const float w7 = w_oi[7];
    const float w8 = w_oi[8];

    // w=0
    acc[0] += w0 * x_bi[0];

    // w=1
    acc[1] += w1 * x_bi[1];
    acc[2] += w1 * x_bi[2];
    acc[3] += w1 * x_bi[3];
    acc[4] += w1 * x_bi[4];

    // w=2
    acc[5] += w2 * x_bi[5];
    acc[6] += w2 * x_bi[6];
    acc[7] += w2 * x_bi[7];
    acc[8] += w2 * x_bi[8];
    acc[9] += w2 * x_bi[9];
    acc[10] += w2 * x_bi[10];
}
```

...

Memory Layout - Flatting & Cont. Vec

- Flattened Q, K vector projections and ensured contiguous memory access.
- Together with blocking significantly improves cache efficiency for GEMM score computation.

q:	7 IPA Blades	5 DAA Blades
-----------	---------------------	---------------------

Memory Layout - Blocking

- Currently only implemented in `equi_geom_attn`
 - Around 2x runtime improvement, more efficient caching
- Coming for `equi_linear` in V1.1
 - We expect massive gains, since the runtime share of `equi_linear` is much larger than `equi_geom_attn`

Future considerations

- Vectorisation
- Automatic FLOPs and Data Movement
- Intel VTune didn't work, perf flags missing (counters missing from silicon)
- We have to change server infra...